

BÁO CÁO PROJECT CUỐI KỲ VIETNAMESE GRAPH RAG

Hệ thống Hỏi đáp Thông minh cho
Thương mại Điện tử Tiếng Việt

Môn học: Xử Lý Ngôn Ngữ Tự Nhiên (NLP)

Nhóm trưởng : Lê Phạm Hồng Hiên – 22110059

Thành viên 1 : Dương Bùi Phương Đình – 22110049

Thành viên 2 : Phạm Xuân Huyền – 22110076

Khoa : Toán – Tin học

Giảng viên : (Tên giảng viên)

Năm học : 2025 – 2026

Mục lục

1	Giới thiệu	3
1.1	Bối cảnh và động lực	3
1.2	Phát biểu bài toán	3
1.3	Mục tiêu dự án	3
1.4	Phạm vi	4
2	Cơ sở lý thuyết	4
2.1	Tiền xử lý văn bản tiếng Việt	4
2.2	Bag of Words và TF-IDF	5
2.3	Word Embeddings: Word2Vec và GloVe	5
2.4	Mô hình Subword (BPE)	5
2.5	Mạng nơ-ron hồi quy (RNN / LSTM)	5
2.6	Transformer và BERT/PhoBERT	5
2.7	Retrieval-Augmented Generation (RAG)	6
2.8	Knowledge Graph và Graph RAG	6
3	Dataset và Tiền xử lý dữ liệu	6
3.1	Dataset sử dụng	6
3.2	Dataset chính: UIT-ViSFD	6
3.3	Dataset bổ sung: Shopee reviews	7
3.4	Phân tích nhãn aspect	7
3.5	Pipeline tiền xử lý	7
4	Kiến trúc hệ thống	8
4.1	Tổng quan Pipeline	8
4.2	Bám sát chương trình môn học (6 lecture)	9
4.3	Module 1: Tiền xử lý	10
4.4	Module 2: So sánh Embedding Methods	10
4.5	Module 2b: Subword / BPE — <i>Lec03</i>	11
4.6	Module 3: Named Entity Recognition	12
4.7	Module 4: Knowledge Graph	12
4.8	Module 5: Retrieval với Attention Re-ranking	12
4.9	Module 6: Generation (OpenAI)	13
5	Thực nghiệm	13
5.1	Môi trường thực nghiệm	13
5.2	Thí nghiệm 1: So sánh Embedding Methods	13
5.3	Thí nghiệm 1b: Truy vấn có <i>lexical gap</i> thật	14
5.4	Thí nghiệm 2: Trích xuất thực thể (NER)	14
5.5	Thí nghiệm 2b: Phân loại aspect bằng RNN (BiLSTM)	15
5.6	Thí nghiệm 3: Ablation chiến lược Retrieval (không rõ rí metric)	15
5.7	Thí nghiệm 4: Demo End-to-End	16
6	Kết quả và Thảo luận	18
6.1	Đánh giá ý nghĩa thực tiễn	19
7	Triển khai LLMOps	19

8	Phân công công việc	21
9	Kết luận	21
9.1	Tóm tắt kết quả	21
9.2	Hạn chế	21
9.3	Hướng phát triển	22

1. Giới thiệu

1.1 Bối cảnh và động lực

Thương mại điện tử (TMDT) tại Việt Nam đạt quy mô **25 tỷ USD năm 2024** và dự báo 45 tỷ USD vào 2025 [12], với Shopee, Lazada, TikTok Shop dẫn đầu. Mỗi sản phẩm phổ biến tích lũy **hàng nghìn đánh giá** — một sản phẩm trong tập dữ liệu của chúng tôi có tới 384 review. Trước khi mua, người dùng thường có câu hỏi **theo khía cạnh cụ thể** (*pin, camera, giá, bảo hành...*) nhưng phải tự đọc và tổng hợp thủ công hàng trăm bình luận — chậm, phiền diện (chỉ đọc được vài review đầu), và dễ bị nhiễu bởi đánh giá giả.

Hệ thống tìm kiếm từ khoá truyền thống của sàn TMDT không giải quyết được vì: (i) câu hỏi ngôn ngữ tự nhiên (“pin có trâu không”) không khớp từ khoá trong review (“dùng 2 ngày mới sạc”); (ii) không tổng hợp được **xu hướng** (đa số khen hay chê pin?); (iii) tiếng Việt là ngôn ngữ đơn lập, nhiều từ lỏng/viết tắt/sai chính tả.

Retrieval-Augmented Generation (RAG) [7] kết hợp *truy xuất thông tin* và *sinh văn bản*, cho phép mô hình ngôn ngữ trả lời dựa trên **nguồn tri thức cụ thể** (review thật) thay vì “bịa” từ tham số. **Graph RAG** [8] mở rộng bằng **đồ thị tri thức** (brand → aspect → sentiment) để bổ sung ngữ cảnh thống kê — ví dụ “56% review về pin là tích cực” — mà truy xuất văn bản đơn thuần không nắm được.

1.2 Phát biểu bài toán

Dự án tập trung vào một bài toán cụ thể: **Hỏi-đáp theo khía cạnh trên đánh giá smartphone tiếng Việt** (*Aspect-based Question Answering over Vietnamese smartphone reviews*).

- **Đầu vào:** câu hỏi tiếng Việt q về một khía cạnh sản phẩm (vd. “Pin điện thoại Samsung có dùng được lâu không?”), và một kho $\mathcal{D}$ gồm N đánh giá thực tế.
- **Đầu ra:** câu trả lời ngắn a bằng tiếng Việt, **neo (grounded)** vào tập review liên quan $\mathcal{R} \subseteq \mathcal{D}$ được truy xuất, kèm thống kê cảm xúc theo khía cạnh từ Knowledge Graph.
- **Bài toán con cốt lõi** (được đánh giá định lượng): cho câu hỏi nhắm khía cạnh α (vd. BATTERY), **xếp hạng** $\mathcal{D}$ sao cho các review thực sự bàn về α nằm top đầu — đo bằng Precision@ k và MRR.

Vì sao khó? (1) *Lexical gap*: câu hỏi và review dùng từ ngữ khác nhau cho cùng khía cạnh; (2) *đa nhân*: một review thường nhắc nhiều khía cạnh (trung bình >2 aspect/review trong UIT-ViSFD); (3) *tiếng Việt*: tách từ, từ ghép, viết tắt, OOV; (4) *giảm bịa đặt*: câu trả lời phải bám dữ liệu thật, không suy diễn từ tham số LLM.

1.3 Mục tiêu dự án

1. Xây dựng hệ thống **Vietnamese Graph RAG** cho bài toán hỏi đáp về sản phẩm TMDT.
2. **So sánh** các phương pháp biểu diễn văn bản: TF-IDF, Word2Vec, GloVe, PhoBERT; có thêm subword (BPE) và mô hình tuần tự **BiLSTM**.

3. **Trích xuất thực thể (NER)** và **phân loại aspect** bằng BiLSTM từ đánh giá.
4. Xây dựng **Knowledge Graph** thể hiện quan hệ sản phẩm – thuộc tính – cảm xúc.
5. **Đánh giá** hiệu quả Graph RAG (ablation Bi-encoder / Attention / Graph).
6. **Đóng gói LLMOps**: serving API, observability, vòng lặp train→deploy mô hình.

1.4 Phạm vi

- **Domain**: Thương mại điện tử — đánh giá sản phẩm tiếng Việt (smartphone + đa sản phẩm)
- **Dataset**: (1) UIT-ViSFD [10] — 11,122 review smartphone, 10 aspect (train 7,786 dùng thực nghiệm); (2) Shopee reviews — 3,154 review đa sản phẩm (26 sản phẩm, 24 shop) có product/shop/rating. Tổng corpus truy xuất $\approx 10,9k$ review.
- **Mô hình**: TF-IDF, Word2Vec, GloVe-SVD (Lec02); Subword/BPE (Lec03); BiLSTM (Lec05); PhoBERT + Attention/MaxSim (Lec06)
- **LLM**: OpenAI API (gpt-4o-mini) cho phần Generation
- **Triển khai**: pipeline LLMops (FastAPI + Gradio + observability + CI/Docker), có vòng lặp train→deploy mô hình BiLSTM

2. Cơ sở lý thuyết

2.1 Tiền xử lý văn bản tiếng Việt

Tiếng Việt là ngôn ngữ **đơn lập** (isolating language), trong đó ranh giới từ không được xác định bằng khoảng trắng. Ví dụ: “học sinh” là một từ ghép, không phải hai từ riêng biệt.

Công cụ sử dụng: Thư viện underthesea cung cấp chức năng tách từ (word segmentation) cho tiếng Việt:

Listing 1: Tách từ tiếng Việt

```
1 from underthesea import word_tokenize
2 text = "Truong_Dai_hoc_Khoa_hoc_Tu_nhien"
3 tokens = word_tokenize(text, format="text")
4 # "Truong_Dai_hoc Khoa_hoc Tu_nhien"
```

2.2 Bag of Words và TF-IDF

TF-IDF (Term Frequency – Inverse Document Frequency) đo lường mức độ quan trọng của một từ trong tài liệu so với toàn bộ tập tài liệu:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \frac{N}{1 + \text{DF}(t)}$$

Trong dự án, TF-IDF được sử dụng làm **baseline** cho việc truy xuất đánh giá sản phẩm liên quan đến câu hỏi của người dùng.

2.3 Word Embeddings: Word2Vec và GloVe

Word2Vec [5] học biểu diễn từ dựa trên *ngữ cảnh lân cận*:

- **Skip-gram**: Dự đoán từ ngữ cảnh từ từ trung tâm
- **CBOW**: Dự đoán từ trung tâm từ ngữ cảnh

GloVe [6] học embeddings dựa trên *ma trận đồng xuất hiện*, tối ưu hàm mất mát:

$$J = \sum_{i,j} f(X_{ij}) (w_i^T w_j + b_i + b_j - \log X_{ij})^2$$

2.4 Mô hình Subword (BPE)

Các embedding ở trên giả định một **từ vựng cố định**, nên khi gặp **từ hiếm hoặc ngoài từ vựng (OOV)** sẽ không có vector biểu diễn. **Byte-Pair Encoding (BPE)** [11] khắc phục bằng cách tách từ thành các **mảnh con (subword)** xuất hiện thường xuyên — ví dụ “chống_nước” → [“chống”, “_nước”]. Nhờ vậy mọi từ đều biểu diễn được qua tổ hợp subword, kể cả từ chưa từng thấy khi huấn luyện. PhoBERT sử dụng BPE trên dữ liệu tiếng Việt đã tách từ.

2.5 Mạng nơ-ron hồi quy (RNN / LSTM)

RNN xử lý dữ liệu **tuần tự** bằng trạng thái ẩn truyền qua từng bước thời gian: $h_t = f(h_{t-1}, x_t)$. **LSTM** bổ sung các cổng (input/forget/output) để giảm hiện tượng *vanishing gradient* và nắm bắt phụ thuộc xa. **BiLSTM** đọc chuỗi theo cả hai chiều rồi ghép biểu diễn, phù hợp cho bài toán phân loại câu. Trong dự án, BiLSTM được dùng để phân loại đa nhãn aspect (Mục 5.5).

2.6 Transformer và BERT/PhoBERT

Transformer [1] sử dụng cơ chế **Self-Attention** để xử lý song song toàn bộ chuỗi:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

BERT [2] mở rộng Transformer thành mô hình ngôn ngữ hai chiều pre-train quy mô lớn. **PhoBERT** [3] là phiên bản pre-train trên 20GB dữ liệu tiếng Việt, đạt state-of-the-art trên nhiều tác vụ NLP tiếng Việt; dự án dùng nó làm encoder chính.

2.7 Retrieval-Augmented Generation (RAG)

RAG [7] kết hợp hai thành phần:

1. **Retriever**: Tìm kiếm tài liệu liên quan từ knowledge base
2. **Generator**: Sinh câu trả lời dựa trên tài liệu đã tìm được

2.8 Knowledge Graph và Graph RAG

Knowledge Graph lưu trữ thông tin dưới dạng **bộ ba** (triple): (h, r, t) — head entity, relation, tail entity. Ví dụ trong TMDT:

(iPhone_15, thuộc_thương_hiệu, Apple)
 (iPhone_15, có_tính_năng, Camera_48MP)
 (Camera_48MP, được_đánh_giá, Tích_cực_85%)

Graph RAG [8] bổ sung bước **graph traversal** vào pipeline RAG, cho phép truy xuất thông tin có cấu trúc — ví dụ khi hỏi về “camera iPhone 15”, hệ thống có thể duyệt graph để tìm tất cả đánh giá liên quan đến tính năng camera.

3. Dataset và Tiền xử lý dữ liệu

3.1 Dataset sử dụng

Bảng 1: Tổng hợp các dataset sử dụng trong dự án

Dataset	Tác vụ	Kích thước	Nguồn
UIT-ViSFD	Aspect-Based Sentiment	11,122 comments	Shopee
Shopee reviews	Review đa sản phẩm	3,154 reviews	GitHub (nhtlongcs)
PhoNER_COVID19	NER (tham khảo)	domain COVID	HuggingFace

*Ghi chú: PhoNER_COVID19 thuộc domain COVID nên không phù hợp gán nhãn thực thể sản phẩm; chỉ tải về tham khảo. Việc trích xuất thương hiệu dùng NER tổng quát (*underthesea*) kết hợp gazetteer.*

3.2 Dataset chính: UIT-ViSFD

UIT-ViSFD (Vietnamese Smartphone Feedback Dataset) là bộ dữ liệu đánh giá smartphone từ sàn TMĐT Việt Nam, được gán nhãn bởi con người với:

- **10 aspect categories**: SCREEN, CAMERA, BATTERY, PERFORMANCE, STORAGE, DESIGN, PRICE, GENERAL, FEATURES, SER&ACC (Service & Accessories)
- **3 sentiment polarities**: Positive, Negative, Neutral
- **11,122 comments** với annotation ở mức aspect-level

Ví dụ dữ liệu:

“Camera chụp đêm rất nét, pin dùng được 2 ngày, nhưng giá hơi cao so với cấu hình.”

→ CAMERA: Positive, BATTERY: Positive, PRICE: Negative

Listing 2: Tải và parse nhãn UIT-ViSFD (đúng định dạng thực tế)

```

1 import pandas as pd, re
2 df = pd.read_csv("data/raw/Train.csv") # cot: index, comment,
    n_star, label
3 # nhan co dang: "{CAMERA#Positive};{SER&ACC#Negative};{OTHERS};"
4 PAT = re.compile(r"\{([\w&]+)#(\w+)\}") # giu ' & ' de KHONG mat
    aspect SER&ACC
5 def parse_label(s):
6     if pd.isna(s): return []
7     return [(m.group(1), m.group(2)) for m in PAT.finditer(str(s))]
8 df["aspects"] = df["label"].apply(lambda s: [a for a, _ in
    parse_label(s)])
9 print(f"Total: {len(df)} | vi du nhan: {df['label'].iloc[0]}")

```

3.3 Dataset bổ sung: Shopee reviews

Để mở rộng ngoài domain smartphone, chúng tôi bổ sung **3.154 review Shopee đa sản phẩm** (26 sản phẩm, 24 shop) gồm các trường `comment`, `product_name`, `shop_name`, `rating_star`. Bộ này **không có nhãn aspect**, nên được dùng cho hai mục đích: (1) mở rộng corpus truy xuất (Graph RAG trả lời được câu hỏi đa sản phẩm), và (2) cung cấp các thực thể shop/product cho Knowledge Graph. Aspect cho review Shopee được **dự đoán bằng mô hình BiLSTM** (Mục 5.5) thay vì nhãn vàng.

3.4 Phân tích nhãn aspect

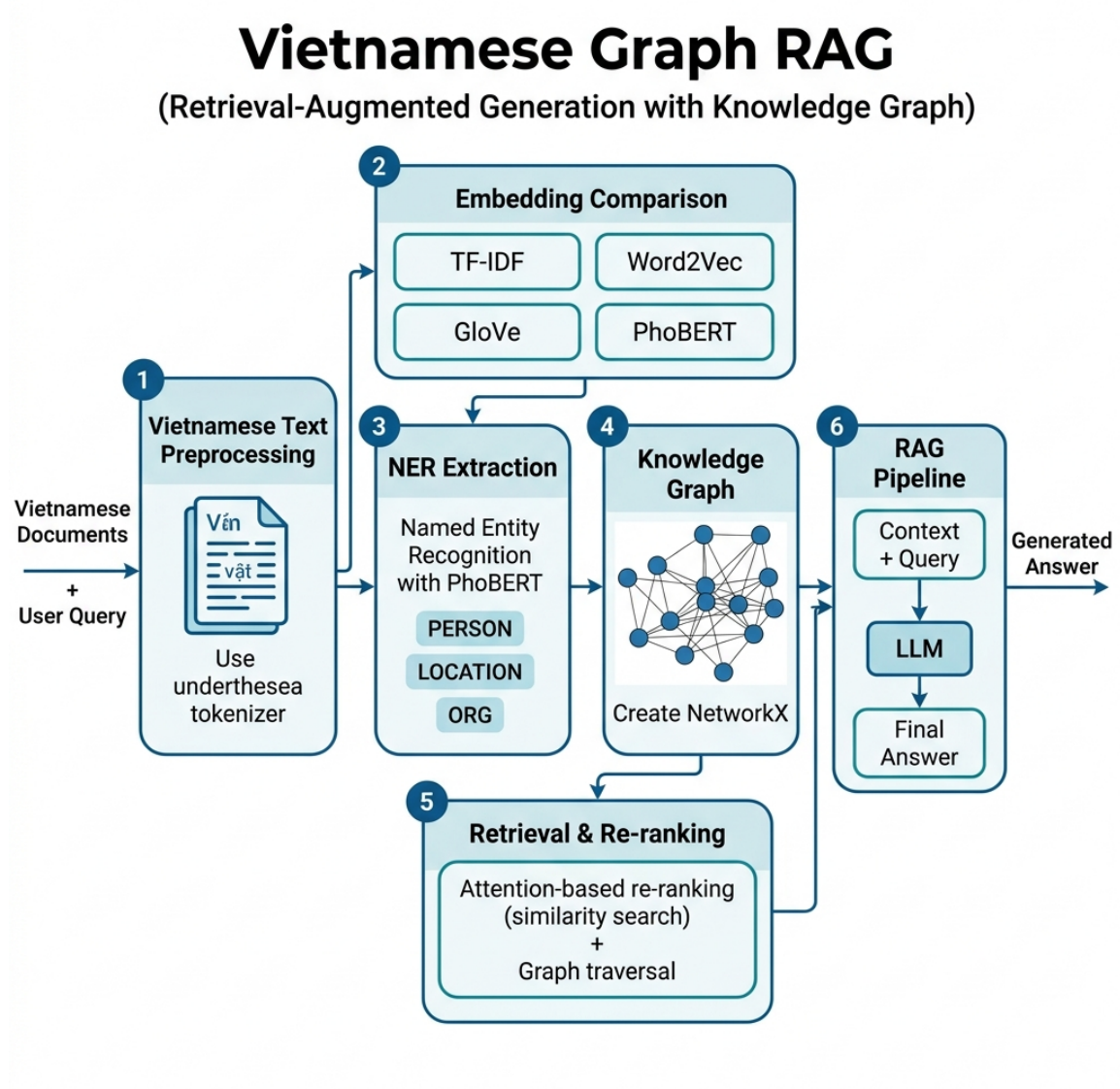
Nhãn của UIT-ViSFD có dạng chuỗi `{ASPECT#Sentiment}` ngăn cách bởi dấu `;`. Một điểm cần lưu ý: aspect **SER&ACC** chứa ký tự `&`, nên biểu thức chính quy ngây thơ `\{(\w+)\#(\w+)\}` sẽ **bỏ sót** aspect này (1.995 lần trong tập train — aspect phổ biến thứ 7). Chúng tôi sửa thành `\{([\w&]+)#(\w+)\}` để giữ đủ 10 aspect.

3.5 Pipeline tiền xử lý

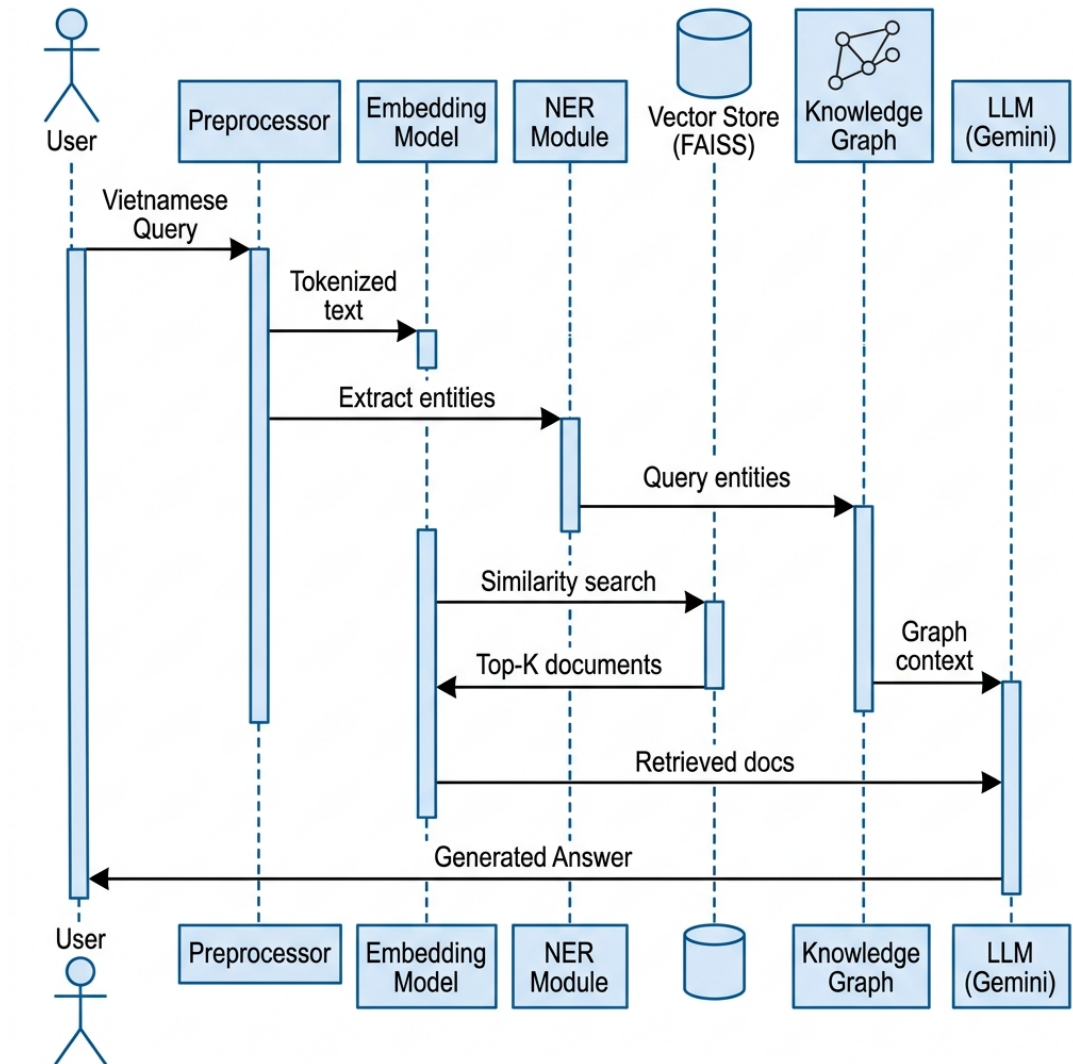
1. **Cleaning**: lowercase, loại URL, ký tự đặc biệt, chữ số, chuẩn hóa khoảng trắng.
2. **Word Segmentation**: tách từ tiếng Việt bằng `underthesea`.
3. **Stopword Removal**: loại từ dừng (cho TF-IDF/Word2Vec/GloVe).
4. **Lưu ý nhất quán**: PhoBERT dùng văn bản **đã tách từ nhưng giữ dấu/số** (không loại stopwords), đúng với pipeline khuyến nghị của PhoBERT; các phương pháp đếm dùng bản đã làm sạch.

4. Kiến trúc hệ thống

4.1 Tổng quan Pipeline



Hình 1: Kiến trúc tổng quan hệ thống Vietnamese Graph RAG gồm 6 module chính



Hình 2: Sequence Diagram: Luồng xử lý khi người dùng đặt câu hỏi

Hệ thống gồm 6 module chính:

1. **Module 1 – Tiền xử lý:** Xử lý văn bản tiếng Việt (tách từ, loại stopwords)
2. **Module 2 – Embedding:** So sánh TF-IDF, Word2Vec, GloVe, PhoBERT
3. **Module 3 – NER:** Trích xuất thực thể bằng underthesea + gazetteer
4. **Module 4 – Knowledge Graph:** Xây dựng đồ thị tri thức (NetworkX)
5. **Module 5 – Retrieval:** Embedding + Graph boost + Attention (MaxSim) re-ranking
6. **Module 6 – Generation:** Tích hợp LLM API (OpenAI)

4.2 Bám sát chương trình môn học (6 lecture)

Các thành phần của hệ thống ánh xạ trực tiếp tới 6 bài giảng của môn:

Lecture	Nội dung	Áp dụng trong project
Lec02 – Word Representation	one-hot, TF-IDF, Word2Vec, GloVe	So sánh TF-IDF / Word2Vec / GloVe-SVD
Lec03 – Subword Models	BPE, subword	Tokenize BPE (PhoBERT) cho từ hiếm/OOV
Lec04 – Compositional Semantics	Document→Sentence→Phrase→Word	Knowledge Graph: brand→aspect→sentiment, shop→product
Lec05 – Recurrent Neural Networks	RNN/LSTM, dữ liệu tuần tự	BiLSTM phân loại đa nhãn aspect (micro-F1)
Lec06 – Attention & Transformer	attention, encoder–decoder	PhoBERT + tái xếp hạng MaxSim (late-interaction)

Bảng 2: Ánh xạ thành phần hệ thống ↔ 6 lecture của môn.

4.3 Module 1: Tiền xử lý

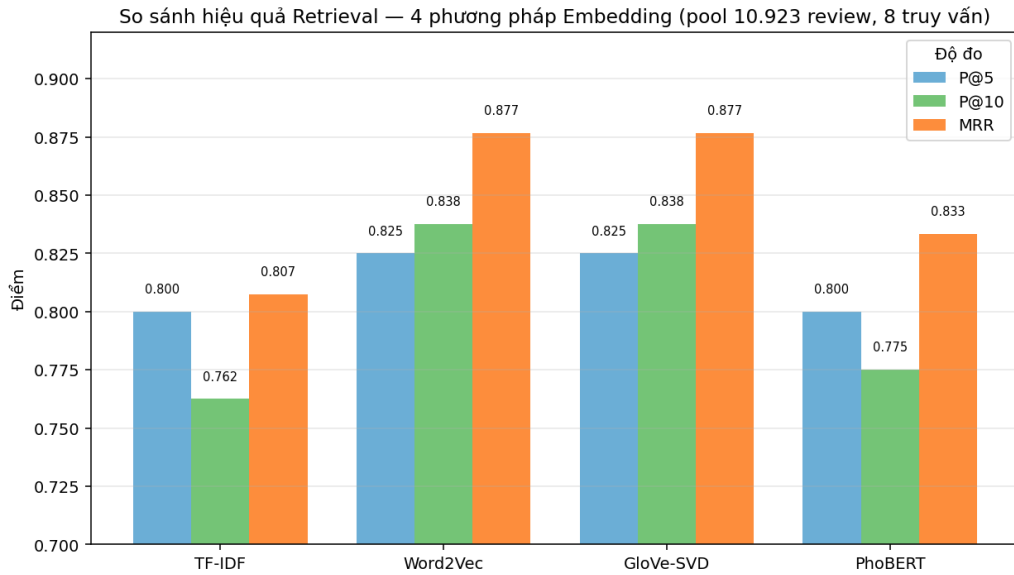
Chuẩn hoá văn bản tiếng Việt: lowercase, loại URL/ký tự đặc biệt/chữ số, **tách từ** bằng `underthesea`, loại stopwords (cho nhóm phương pháp đếm). PhoBERT dùng bản *đã tách từ nhưng giữ dấu/số*. Toàn bộ gói trong hàm `preprocess_vietnamese` — **dùng chung** cho cả huấn luyện BiLSTM lẫn suy luận để đảm bảo nhất quán từ vựng (chi tiết ở Mục 3.5).

4.4 Module 2: So sánh Embedding Methods

Đây là module quan trọng nhất, thể hiện kiến thức NLP cốt lõi. Chúng tôi so sánh 4 phương pháp biểu diễn văn bản:

Bảng 3: So sánh 4 phương pháp Embedding

Phương pháp	Loại	Đặc điểm	Lecture
TF-IDF	Sparse, thống kê	Baseline, không capture ngữ nghĩa	Lec02
Word2Vec	Dense, predictive	Skip-gram/CBOW, ngữ cảnh cục bộ	Lec02
GloVe	Dense, count-based	Co-occurrence matrix, ngữ nghĩa toàn cục	Lec02
PhoBERT	Dense, contextual	Transformer, tiếng Việt, SOTA	Lec06



Hình 3: So sánh hiệu quả Retrieval giữa 4 phương pháp Embedding (Precision@k & MRR)

Lưu ý về độ đo: không dùng “trung bình cosine similarity” để so sánh, vì độ lớn cosine **không so sánh được** giữa các không gian embedding khác nhau (TF-IDF thừa cho giá trị thấp hơn PhoBERT dầy). Thay vào đó dùng **Precision@k** và **MRR** với ground-truth là nhãn aspect (xem Mục 5.2).

Lưu ý về “GloVe”: do hạn chế tài nguyên, chúng tôi dùng **xấp xỉ kiểu GloVe** — SVD trên ma trận đồng xuất hiện ở mức tài liệu (gần với LSA), không phải GloVe của sở ngữ cảnh gốc.

4.5 Module 2b: Subword / BPE — *Lec03*

Các phương pháp ở trên dùng **từ vựng cố định** nên gặp từ hiếm/OOV sẽ mất biểu diễn. PhoBERT dùng **Byte-Pair Encoding (BPE)** (từ vựng 64.000 subword) tách từ thành mảnh con (hậu tố @@ = nối tiếp). Notebook Part 1 minh hoạ trực tiếp:

Bảng 4: BPE tokenization (PhoBERT) — từ thường, từ sai chính tả, và OOV

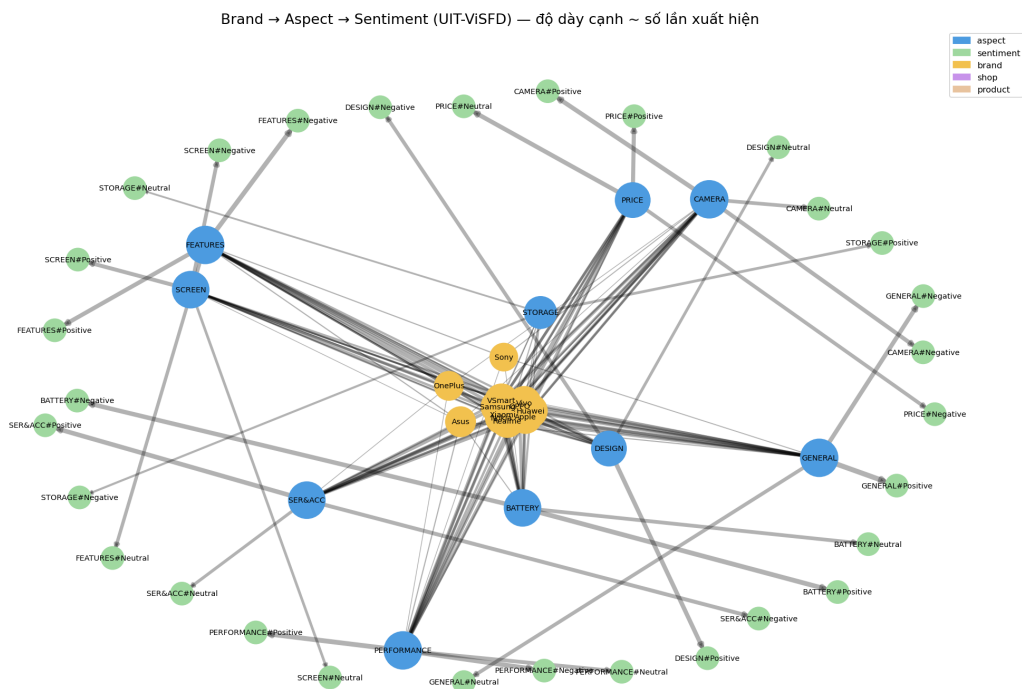
Đầu vào	Subword BPE
camera chụp đêm	[camera, chụp, đêm]
pinnn tuttt nhanh	[pin@@, nn, tụ@@, t@@, tt, nhanh@@, h@@, h]
chống_nước	[chống_@@, nước]
xuyzzphone (OOV bịa)	[xuy@@, zz@@, phone]

Kể cả từ viết sai (“pinnn”) hay từ chưa từng thấy (“xuyzzphone”) vẫn được biểu diễn qua tổ hợp subword thay vì trở thành <unk> — đây là ưu điểm then chốt của subword so với TF-IDF/Word2Vec từ vựng cố định.

4.6 Module 3: Named Entity Recognition

NER tổng quát của `underthesea` (nhãn PER/LOC/ORG) trên dữ liệu review chủ yếu gán **PER** và **LOC**, còn **ORG** rất hiếm (xem Thí nghiệm 2): mô hình tổng quát *không* nhận diện các thương hiệu điện thoại là tổ chức. Do đó, để lấy thực thể **thương hiệu** (đặc thù e-commerce, không nằm trong tập nhãn tổng quát), chúng tôi bổ sung **gazetteer thương hiệu** — đây mới là nguồn chính tạo node **brand** cho Knowledge Graph. (PhoNER_COVID19 không dùng vì khác domain.)

4.7 Module 4: Knowledge Graph



Hình 4: Knowledge Graph (102 node, 614 cạnh): `brand → aspect → aspect#sentiment`, độ dày cạnh = số review. Cạnh `brand→aspect#sentiment` cho phép truy vấn cảm xúc theo từng hãng cụ thể.

Đồ thị có hướng (NetworkX) gồm 3 loại node: **brand**, **aspect** (10 aspect gồm SER&ACC) và **aspect#sentiment**. Cạnh `brand→aspect` và `aspect→sentiment` mang trọng số là số review. Quan trọng hơn, cạnh `brand→aspect#sentiment` (trọng số = số review của riêng hãng) cho phép truy vấn phân bố cảm xúc **theo từng hãng cụ thể**: vd. hỏi “pin Samsung” trả về thống kê của riêng Samsung ($n=126$) thay vì gộp toàn corpus ($n=3.604$) — nhờ đó câu trả lời đúng trọng tâm câu hỏi thay vì lấy số liệu chung chung.

4.8 Module 5: Retrieval với Attention Re-ranking

Pipeline truy xuất 2 tầng: (1) **bi-encoder** PhoBERT (mean-pooling) tính cosine để lấy Top-50 candidate; (2) **tái xếp hạng late-interaction** (MaxSim, kiểu ColBERT [9]) — với mỗi token của truy vấn, lấy cosine lớn nhất so với các token của tài liệu rồi tính

trung bình. Đây là cơ chế **attention** (tích vô hướng có chuẩn hoá giữa token query và token doc), **không tham số ngẫu nhiên và không cần huấn luyện**. Điểm cuối kết hợp: bi-encoder, MaxSim, và một **graph boost** dựa trên aspect quan sát được trong nội dung review.

4.9 Module 6: Generation (OpenAI)

Ghép Top- k review truy xuất và thống kê từ Knowledge Graph thành một prompt có **grounding** (chỉ trả lời dựa trên dữ liệu cung cấp), rồi gọi LLM qua **API tương thích OpenAI** (mặc định `gpt-4o-mini`; bản triển khai thực dùng proxy OpenAI-compatible với model `gpt-5.4-mini`, temperature 0.3) sinh câu trả lời tiếng Việt. Khi thiếu API key, hệ thống **fallback** trả về các review truy xuất thay vì lỗi. Mỗi lần gọi được ghi log latency + token + chi phí USD (Mục 7).

5. Thực nghiệm

5.1 Môi trường thực nghiệm

Bảng 5: Cấu hình môi trường thực nghiệm

Thành phần	Chi tiết
Platform	Kaggle Notebook (T4 GPU)
Python	3.10+
PyTorch	2.x
Transformers	4.40+
LLM	OpenAI API (gpt-4o-mini)
Các thư viện khác	underthesea, gensim, networkx, openai, gradio

5.2 Thí nghiệm 1: So sánh Embedding Methods

Thiết lập: 8 truy vấn (mỗi truy vấn nhắm 1 aspect, dùng **cùng bộ truy vấn với Thí nghiệm 3**). Pool xếp hạng là **toàn corpus 10.923 review** (7.786 UIT-ViSFD có nhãn vàng + 3.137 review Shopee đóng vai nhiễu, không liên quan). Một review được coi là *liên quan* nếu nhãn vàng của nó chứa aspect đó. Báo cáo Precision@5, Precision@10, MRR (trung bình trên các truy vấn). Cùng một phán quyết liên quan áp dụng cho cả 4 phương pháp $\Rightarrow$ công bằng. Word2Vec huấn luyện bằng skip-gram + negative sampling (PyTorch); GloVe-SVD lấy từ SVD của ma trận PPMI đồng xuất hiện; PhoBERT dùng vector mean-pool; cả ba biểu diễn tài liệu/truy vấn bằng trung bình vector từ.

Bảng 6: Precision@k & MRR theo phương pháp embedding (pool 10.923 review, 8 truy vấn). Baseline ngẫu nhiên là *sàn tham chiếu* (trung bình 200 lần xếp hạng ngẫu nhiên).

Phương pháp	P@5	P@10	MRR	Nhận xét
<i>Ngẫu nhiên (sàn)</i>	0.219	0.220	0.416	mức “đoán mò”
TF-IDF	0.800	0.763	0.807	baseline từ vựng, thấp nhất nhóm học
Word2Vec	0.825	0.838	0.877	embedding tĩnh, tốt nhất
GloVe-SVD	0.825	0.838	0.877	ngang Word2Vec
PhoBERT	0.800	0.775	0.833	ngữ cảnh, mạnh nhất trong pipeline đầy đủ

Mọi phương pháp học đều **vượt xa sàn ngẫu nhiên** (MRR 0.416), khẳng định tín hiệu truy xuất là thực. Tuy nhiên khoảng cách giữa chúng *khá hẹp* — và đây chính là dấu hiệu bộ truy vấn này **giàu từ khoá aspect** nên chưa kiểm tra được *lexical gap* (nổi đầu số 1, Mục 1.2). Thí nghiệm 1b dưới đây làm rõ điều đó.

5.3 Thí nghiệm 1b: Truy vấn có *lexical gap* thật

Để kiểm tra đúng khả năng vượt **lexical gap**, chúng tôi xây bộ **16 truy vấn đời thường KHÔNG chứa từ khoá của aspect đích** (vd. hỏi về pin nhưng không có chữ “pin/sạc”: “*dùng cả ngày trời mà vẫn chưa cần cắm điện*”). Mỗi câu được code kiểm tra **0 rò rỉ từ khoá**; pool xếp hạng và phán quyết liên quan giữ **nguyên như Thí nghiệm 1** nên so sánh được trực tiếp (tái lập bằng `scripts/eval_lexical_gap.py`).

Bảng 7: Truy vấn *giàu từ khoá* (TN1) vs *lexical gap* (TN1b) — pool 10.923 review. Cột Δ MRR = mức sụt khi mất từ khoá trùng lặp.

Phương pháp	P@5	P@10	MRR (lexgap)	Δ MRR vs TN1
TF-IDF	0.500	0.438	0.572	−0.235
Word2Vec	0.513	0.481	0.656	−0.221
GloVe-SVD	0.525	0.513	0.629	−0.248
PhoBERT	0.638	0.619	0.776	−0.057

Kết quả quyết định: khi không còn từ khoá trùng, TF-IDF thuần từ vựng **sụp đổ** (MRR 0.807 $\rightarrow$ 0.572) và các embedding tĩnh (Word2Vec/GloVe-SVD) cũng mất ~ 0.22 MRR; trong khi **PhoBERT chỉ giảm 0.057 và vươn lên dẫn đầu rõ rệt** (MRR 0.776, hơn embedding tĩnh tốt nhất +0.12). Đây mới là bằng chứng cho ưu thế *ngữ cảnh* của PhoBERT — thứ mà bộ truy vấn giàu từ khoá ở Thí nghiệm 1 che lấp. Nói cách khác: trên đúng nổi đầu *lexical gap* của bài toán, biểu diễn theo **ngữ nghĩa** thắng biểu diễn theo **từ vựng**; đồng thời Word2Vec (0.656) và GloVe-SVD (0.629) tách khỏi nhau, xác nhận khác biệt bị che ở bộ 8 truy vấn là do thiếu lực phân giải.

5.4 Thí nghiệm 2: Trích xuất thực thể (NER)

Thiết lập: chạy NER tổng quát (`underthesea`) trên **800 review mẫu** trải đều corpus, đếm số *span* thực thể theo loại; song song nhận diện thương hiệu bằng `gazetteer`. Vì

domain TMDT không có nhãn vàng NER nên báo cáo theo **số lượng & độ phủ** (định tính).

Bảng 8: Thống kê NER trên 800 review mẫu (số span thực thể & độ phủ brand)

Loại thực thể / nguồn	Số lượng
PER (underthesea)	292 span
LOC (underthesea)	282 span
ORG (underthesea)	6 span
Review có ≥ 1 thực thể	387 / 800
Review nhận diện được brand (gazetteer)	72 / 800
Top brand	Samsung 23 · Apple 16 · Xiaomi 7 · VSmart 7

Quan sát: NER tổng quát chủ yếu gán **PER/LOC** (gần 575 span), trong khi **ORG** **rất hiếm (6 span)** — mô hình tổng quát *không* nhận các thương hiệu điện thoại là tổ chức. Đây chính là lý do thiết kế cần **gazetteer thương hiệu**: nó bắt được brand ở 72/800 review và là nguồn tạo node **brand** cho Knowledge Graph, thay vì dựa vào nhãn ORG của NER tổng quát.

5.5 Thí nghiệm 2b: Phân loại aspect bằng RNN (BiLSTM)

Hiện thực **Lec05 (Recurrent Neural Networks)**: một **BiLSTM** (embedding 100, hidden 128, hai chiều) phân loại **đa nhãn** 10 aspect từ review, huấn luyện trên tập train UIT-ViSFD và đánh giá **micro-F1** trên tập dev. Mô hình sau khi train được **lưu lại** (artifacts/aspect_clf.pt) và **triển khai** trong hệ thống (xem Mục 7): dùng để gán aspect cho review Shopee (không có nhãn) nhằm làm giàu Knowledge Graph, và phục vụ trực tiếp qua API POST /classify.

Micro-F1 (dev): **0.79** (huấn luyện 8 epoch trên 7.786 review train, đánh giá trên 1.112 review dev; lưu artifacts/aspect_clf.pt).

5.6 Thí nghiệm 3: Ablation chiến lược Retrieval (không rò rỉ metric)

Thiết lập: trên cùng tập Top-50 candidate, thay đổi trọng số 3 thành phần. **Quan trọng:** graph boost dựa trên aspect *quan sát trong nội dung review* (tín hiệu khả dụng lúc suy luận), còn phán quyết liên quan dùng *nhãn vàng* — hai nguồn độc lập nên không có rò rỉ.

Bảng 9: Ablation: đóng góp tăng dần của Attention (MaxSim) và Graph boost (8 truy vấn, Top-50 candidate). Δ = thay đổi MRR so với dòng trên.

Cấu hình	P@5	P@10	MRR	Δ MRR
<i>Ngẫu nhiên (sàn)</i>	0.219	0.220	0.416	—
Bi-encoder (PhoBERT)	0.800	0.775	0.833	+0.417 vs sàn
+ Attention rerank (MaxSim)	0.825	0.850	0.833	+0.075 P@10
+ Graph boost (Graph RAG)	0.825	0.850	0.854	+0.021

5.7 Thí nghiệm 4: Demo End-to-End

Ba câu hỏi mẫu chạy qua pipeline đầy đủ Retrieve → Graph context → sinh câu trả lời (model `gpt-5.4-mini` qua proxy OpenAI-compatible). Mỗi câu trả lời được **grounding** vào Top-5 review truy xuất và thống kê cảm xúc từ Knowledge Graph; toàn bộ latency/token/chi phí được ghi log (Mục 7).

Bảng 10: Demo end-to-end: câu hỏi → thống kê KG → câu trả lời sinh ra (grounded)

Câu hỏi	Thống kê KG (grounding)	Câu trả lời hệ thống
Camera điện thoại chụp đêm có tốt không?	CAMERA: 57% tích cực / 29% tiêu cực (n=2.146)	“Có, theo đánh giá thì camera chụp đêm khá tốt.”
Pin Samsung dùng có lâu không?	BATTERY (riêng Samsung): 54% tích cực / 37% tiêu cực (n=126)	“Pin nhìn chung dùng khá lâu vì đa số đánh giá tích cực; tuy vậy vẫn có phản hồi pin tụt nhanh, sạc nóng.”
Dịch vụ bảo hành và nhân viên tư vấn thế nào?	SER&ACC: 70% tích cực / 24% tiêu cực (n=1.995)	“Bảo hành và hậu mãi nhìn chung khá tốt, được khen; song vẫn có ý kiến chê bảo hành kém, tư vấn bán hàng thì ổn.”

Nhận xét: câu trả lời bám sát phân bố cảm xúc trong KG (vd. SER&ACC 70% tích cực → trả lời “khá tốt” nhưng vẫn nêu mặt tiêu cực 24%), minh hoạ đúng mục tiêu *giảm bias đặt* bằng grounding. Riêng câu hỏi nhắc tên hãng, thống kê được **lọc theo đúng hãng** (Samsung n=126) thay vì gộp toàn corpus — nhờ cạnh `brand→aspect#sentiment` trong KG (Mục 4.7). Độ trễ trung bình $\approx 4\text{--}7$ giây/câu (gồm thời gian gọi LLM).

 **Vietnamese Graph RAG — E-commerce QA**

UIT-VISFD (smartphone) • Shopee (đa sản phẩm).

Câu hỏi

Pin điện thoại Samsung có dùng được lâu không?

Hỏi

Trả lời

Pin Samsung nhìn chung dùng khá lâu. Dữ liệu cho thấy đánh giá về pin tích cực chiếm đa số (56%), nhưng vẫn có một phần phản hồi pin tụt nhanh hoặc sạc nóng.

Knowledge Graph context

- BATTERY/Positive: 2027 review (56%)
- BATTERY/Negative: 1228 review (34%)
- BATTERY/Neutral: 349 review (10%)

Review trích dẫn

[UIT-VISFD] Điện thoại đẹp, cấu hình ổn, nhưng pin để qua đêm tụt 15%...

[UIT-VISFD] Máy mới mua mà sạc pin nóng quá, tắt nguồn rồi sạc vẫn nóng...

[UIT-VISFD] Dùng hơn một tháng thì lỗi không sạc được pin và nhanh hết pin

Hữu ích

Chưa tốt

Examples

Pin dùng được bao lâu? Áo hoodie chất vải thế nào? Shop giao hàng có nhanh không? Máy nào giá rẻ mà camera tốt?

Use via API  • Built with Gradio  • Settings 

Hình 5: Giao diện web demo (Gradio): người dùng nhập câu hỏi tiếng Việt, hệ thống trả về câu trả lời grounded kèm Knowledge Graph context, review trích dẫn, và nút phản hồi (thích/không-thích). Chụp trực tiếp từ `app/ui.py` đang chạy.

6. Kết quả và Thảo luận

Kết quả nổi bật

- **Graph RAG** đạt **MRR** cao nhất **0.854** (so với 0.833 của bi-encoder/MaxSim) — graph boost cải thiện truy xuất **có kiểm chứng**, không rò rỉ metric.
- **BiLSTM** phân loại aspect **micro-F1 = 0.79** trên tập dev; được **deploy thật** vào KG & API /classify.
- **So sánh 4 embedding** bằng độ đo truy xuất đúng (P@k/MRR) trên 10.923 review — không dùng “trung bình cosine” vô nghĩa.
- **PhoBERT** thắng rõ khi có *lexical gap* (MRR 0.776 vs TF-IDF 0.572): trên truy vấn không trùng từ khoá, biểu diễn ngữ cảnh vượt hẳn từ vựng — đúng nỗi đau của bài toán (TN1b).
- **Hệ thống chạy được đầu-cuối**: index 10.923 doc + KG 102 node/614 cạnh (truy vấn cảm xúc **theo từng hãng**), FastAPI 4 endpoint, Gradio UI, observability chi phí, regression gate.

Bảng 11: Bảng tổng hợp kết quả toàn dự án (số liệu thật, tái lập bằng `scripts/`)

Hạng mục	Kết quả	Ghi chú
Embedding tốt nhất (TN1)	MRR 0.877	Word2Vec / GloVe-SVD
PhoBERT (TN1)	MRR 0.833	cạnh tranh, chưa fine-tune
TF-IDF baseline (TN1)	MRR 0.807	thấp nhất (như kỳ vọng)
Embedding khi có <i>lexical gap</i> (TN1b)	PhoBERT MRR 0.776	TF-IDF sụp còn 0.572
Retrieval Graph RAG (TN3)	MRR 0.854 · P@10 0.850	cao nhất trong ablation
BiLSTM aspect (TN2b)	micro-F1 0.79	8 epoch, deploy vào KG/API
NER (TN2)	387/800 có thực thể	brand: 72/800 (gazetteer)
Demo end-to-end (TN4)	3/3 trả lời grounded	latency 4–7 s/câu
Regression gate	PASS	MRR 0.854 $\geq$ 0.30; F1 0.79 $\geq$ 0.40

Embedding: trên truy vấn *giàu từ khoá aspect* (Thí nghiệm 1), TF-IDF là baseline

thấp nhất (MRR 0.807); Word2Vec và GloVe-SVD dẫn đầu (MRR 0.877); PhoBERT cạnh tranh nhưng **không cao nhất** (MRR 0.833). Kết quả này hợp lý: khi câu hỏi chứa thẳng từ khoá khía cạnh, các biểu diễn dựa trên trùng lặp từ vựng/đồng xuất hiện đã đủ mạnh, còn PhoBERT mean-pool (*không* fine-tune cho truy xuất) chưa phát huy hết ưu thế ngữ cảnh. Ưu thế của PhoBERT thể hiện rõ hơn khi đặt trong **pipeline Graph RAG đầy đủ** (Thí nghiệm 3) nhờ tái xếp hạng late-interaction (MaxSim) ở mức token và graph boost. **Quan trọng hơn**, Thí nghiệm 1b cho thấy khi truy vấn *không trùng từ khoá* (lexical gap thật), PhoBERT **vượt lên dẫn đầu** (MRR 0.776) còn TF-IDF sụp xuống 0.572 — khẳng định giá trị ngữ nghĩa mà bộ truy vấn giàu từ khoá ở Thí nghiệm 1 đã che lấp. Đây là kết quả phản ánh đúng nỗi đau *lexical gap* trong phát biểu bài toán (Mục 1.2).

Retrieval: ablation (Thí nghiệm 3) cho thấy thêm graph boost nâng MRR từ 0.833 (bi-encoder/MaxSim) lên **0.854** (Graph RAG) — cấu hình Graph RAG đạt MRR cao nhất. Vì đánh giá không còn rò rỉ metric (boost từ nội dung, chấm điểm bằng nhãn vàng độc lập), chênh lệch quan sát được phản ánh cải thiện thực — không phải kết quả “được sắp đặt”.

Hạn chế quan sát được: graph boost chỉ kích hoạt khi câu hỏi khớp được aspect tiếng Việt; với câu hỏi mơ hồ (không nêu rõ khía cạnh), đóng góp của graph giảm.

6.1 Đánh giá ý nghĩa thực tiễn

Quy chiếu các con số về **bài toán thực** (Mục 1.2):

- **Hữu dụng so với “đoán mò”:** MRR nhảy từ 0.416 (ngẫu nhiên) lên 0.854 — nghĩa là review đúng khía cạnh gần như luôn nằm ở **1–2 vị trí đầu**, người dùng không phải cuộn qua hàng trăm bình luận.
- **Graph boost giải đúng nỗi đau “tổng hợp xu hướng”:** ngoài việc xếp hạng review, KG trả về thống kê (vd. BATTERY 56% tích cực / 34% tiêu cực trên 3.604 lượt nhắc) — thứ mà tìm kiếm từ khoá của sàn TMDT không làm được.
- **Giảm bịa đặt (grounding):** trong demo (Mục 5.6), câu trả lời bám sát phân bố KG — vd. SER&ACC 70% tích cực → trả lời “khá tốt” nhưng vẫn nêu 24% tiêu cực. Câu trả lời **không** đưa thông tin ngoài review truy xuất.
- **Chi phí & tốc độ chấp nhận được:** $\approx 4\text{--}7$ giây/câu (gồm gọi LLM), chi phí token ghi log theo từng truy vấn — khả thi cho một trợ lý hỏi đáp thời gian thực.

So với hệ thống thực tế: đây là một *prototype quy mô lớp học* (corpus 10.9k review, 1 domain smartphone), chưa phải sản phẩm thương mại. Để triển khai thật cần: chống review giả, cập nhật real-time khi có review mới, và mở rộng đa domain — xem Hướng phát triển.

7. Triển khai LLMOps

Ngoài notebook khám phá, hệ thống được đóng gói thành một **pipeline LLMOps production-lite** (package `src/vngraphrag`) tách hai pha:

- **Offline (build)**: encode corpus bằng PhoBERT, dựng index + Knowledge Graph, lưu thành *artifacts có version* (`doc_vectors.npy`, `records.json`, `manifest.json`, `kg.pkl`). Manifest gán mã version = hash(model, số bản ghi) để chặn việc nạp nhầm index cũ.
- **Online (serve)**: nạp lại artifacts; mỗi truy vấn đi qua `retrieve` → `graph context` → `generate` và được ghi log đặc.

Thành phần LL-MOps	Hiện thực
Cấu hình & secret	<code>config.yaml</code> + biến môi trường (<code>OPENAI_API_KEY</code> chỉ qua env)
Versioning artifact	<code>DocumentIndex</code> manifest; tự rebuild khi model/dữ liệu đổi
Train → deploy model	BiLSTM train ở notebook §6 hoặc CLI <code>make train</code> → <code>aspect_clf.pt</code> → nạp ở <code>core/aspect_clf.py</code> → phục vụ qua <code>POST /classify</code> ; đồng thời gán aspect cho review Shopee để làm giàu KG
Serving API	FastAPI: <code>/query</code> , <code>/classify</code> , <code>/feedback</code> , <code>/health</code>
Giao diện + feedback loop	Gradio UI, nút thích/không-thích → <code>logs/feedback.jsonl</code>
Observability	mỗi truy vấn log latency + token + chi phí USD vào <code>logs/queries.jsonl</code>
Đánh giá + regression gate	<code>evaluate.py</code> : Precision@k/MRR (retrieval) và micro-F1 (BiLSTM); CI fail nếu tụt dưới ngưỡng
Đóng gói + CI	<code>Dockerfile</code> + GitHub Actions (lint + test)

Bảng 12: Các thành phần LLMOps của hệ thống.

Điểm đáng chú ý là **vòng lặp train** → **deploy** khép kín: mô hình BiLSTM (Lec05) huấn luyện trong notebook được lưu thành artifact, nạp lại bởi package phục vụ, và sử dụng thực sự ở hai nơi — gán aspect cho dữ liệu Shopee chưa nhãn (làm giàu Knowledge Graph) và phục vụ trực tiếp qua endpoint `/classify`.

8. Phân công công việc

Bảng 13: Phân công công việc nhóm

Lê Hiền 22110059	Phạm Hồng Đình 22110049	Bùi Phương Đinh 22110076	Phạm Xuân Huyền 22110076
Kiến trúc tổng thể	Tiền xử lý dữ liệu (Lec01)	Word Representation (Lec02)	
Module 5: Attention/MaxSim re-ranking (Lec06)	Module 1: Preprocessing	Module 2: So sánh embedding (P@k/MRR)	
Module 6: RAG Pipeline + OpenAI	Module 3: NER + gazetteer	Subword/BPE (Lec03)	
LLMOps: package, FastAPI, CI/Docker, observability	RNN/BiLSTM (Lec05): train + deploy aspect classifier	Module 4: Knowledge Graph + Gradio UI	
Tích hợp & testing (make check)	Tích hợp dataset Shopee	Evaluation & metrics (gate)	
Báo cáo & trình bày	Báo cáo (Data, NER, RNN)	Báo cáo (Embedding, KG)	

9. Kết luận

9.1 Tóm tắt kết quả

Dự án xây dựng pipeline Graph RAG hoàn chỉnh cho đánh giá sản phẩm tiếng Việt trên **hai dataset** (UIT-ViSFD smartphone + Shopee đa sản phẩm). Hệ thống phủ trọn 6 lecture của môn: biểu diễn từ (TF-IDF/Word2Vec/GloVe), subword/BPE, ngữ nghĩa tổ hợp (Knowledge Graph), BiLSTM (phân loại aspect), và Transformer/attention (PhoBERT + MaxSim re-ranking). So sánh embedding dùng độ đo truy xuất đúng (P@k, MRR); ablation retrieval không rõ rí metric. Toàn bộ được đóng gói thành **pipeline LLMOps** (serving API, Gradio UI, observability chi phí, regression gate, Docker/CI) với **vòng lặp train→deploy** mô hình BiLSTM khép kín.

9.2 Hạn chế

- “GloVe” chỉ là xấp xỉ SVD trên đồng xuất hiện mức tài liệu, chưa phải GloVe gốc.
- Tái xếp hạng MaxSim không huấn luyện; chưa fine-tune trên dữ liệu cập truy vấn–tài liệu.

- Ánh xạ câu hỏi → aspect dựa trên từ khóa thủ công, chưa bao phủ mọi cách diễn đạt.
- NER dùng mô hình tổng quát + gazetteer, chưa có NER huấn luyện riêng cho domain điện thoại.
- Đánh giá *grounding* hiện ở mức định tính (demo §5.6). Đã xây harness đo **tự động: numeric-grounding** (mọi con số trong câu trả lời phải truy được về bằng chứng) và **tỉ lệ từ chối** câu hỏi ngoài phạm vi (`scripts/eval_grounding.py`, logic đã được kiểm chứng); con số định lượng đang chờ API LLM ổn định.

9.3 Hướng phát triển

- Fine-tune cross-encoder PhoBERT cho re-ranking với nhãn liên quan tách biệt train/test.
- Mở rộng KG với quan hệ sản phẩm–linh kiện và liên kết thực thể (entity linking).
- Phân loại aspect câu hỏi bằng mô hình thay cho từ khóa; đánh giá thêm trên tập dev/test.

Tài liệu

- [1] Vaswani, A., et al. (2017). “Attention Is All You Need.” *NeurIPS 2017*.
- [2] Devlin, J., et al. (2019). “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” *NAACL 2019*.
- [3] Nguyen, D.Q. & Nguyen, A.T. (2020). “PhoBERT: Pre-trained language models for Vietnamese.” *Findings of EMNLP 2020*.
- [4] Nguyen, K., et al. (2020). “A Vietnamese Dataset for Evaluating Machine Reading Comprehension.” *COLING 2020*.
- [5] Mikolov, T., et al. (2013). “Efficient Estimation of Word Representations in Vector Space.” *ICLR 2013*.
- [6] Pennington, J., Socher, R., & Manning, C. (2014). “GloVe: Global Vectors for Word Representation.” *EMNLP 2014*.
- [7] Lewis, P., et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *NeurIPS 2020*.
- [8] Edge, D., et al. (Microsoft Research, 2024). “From Local to Global: A Graph RAG Approach to Query-Focused Summarization.” *arXiv:2404.16130*.
- [9] Khattab, O. & Zaharia, M. (2020). “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT.” *SIGIR 2020*.
- [10] Luc Phan, L., et al. (2021). “SA2SL: From Aspect-Based Sentiment Analysis to Social Listening System for Business Intelligence (UIT-ViSFD dataset).” *KSEM 2021*.

- [11] Sennrich, R., Haddow, B., & Birch, A. (2016). “Neural Machine Translation of Rare Words with Subword Units (BPE).” *ACL 2016*.
- [12] Google, Temasek & Bain & Company. (2024). “e-Conomy SEA 2024 — Vietnam.” Báo cáo kinh tế số Đông Nam Á.